

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

ПРАКТИЧЕСКАЯ РАБОТА 4

По дисциплине Проектирование и реализация баз данных

Тема работы Создание объектов БД

Обучающийся Пухарев Сергей Валерьевич

Факультет факультет прикладной информатики

Группа К3220

Направление подготовки 11.03.02 Инфокоммуникационные технологии и
системы связи

Образовательная программа Программирование в инфокоммуникационных
системах

Обучающийся	_____	_____	<u>Пухарев С.В.</u>
	(дата)	(подпись)	(Ф.И.О.)
Руководитель	_____	_____	<u>Войтюк Т.Е.</u>
	(дата)	(подпись)	(Ф.И.О.)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1 Задание 1	4
2 Задание 2	5
3 Задание 3	6
4 Задание 4	8
5 Задание 5	9
6 Задание 6	11
7 Задание 7	12
ЗАКЛЮЧЕНИЕ	15

ВВЕДЕНИЕ

Целью данной лабораторной работы является изучение объектов БД, а также их создание.

Задачи, решаемые при выполнении работы: знакомство с представлениями, знакомство с функциями и их создание, знакомство с процедурами и их создание.

1 Задание 1

На рисунке 1 представлено окно создания представления.

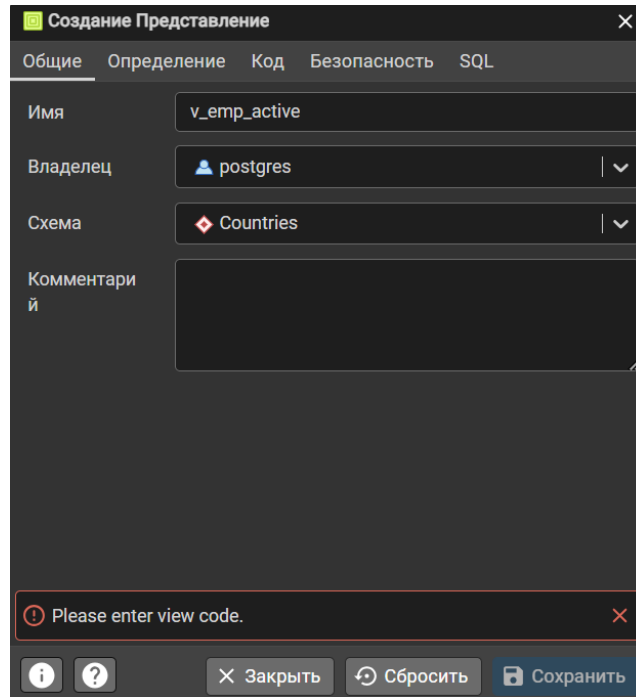
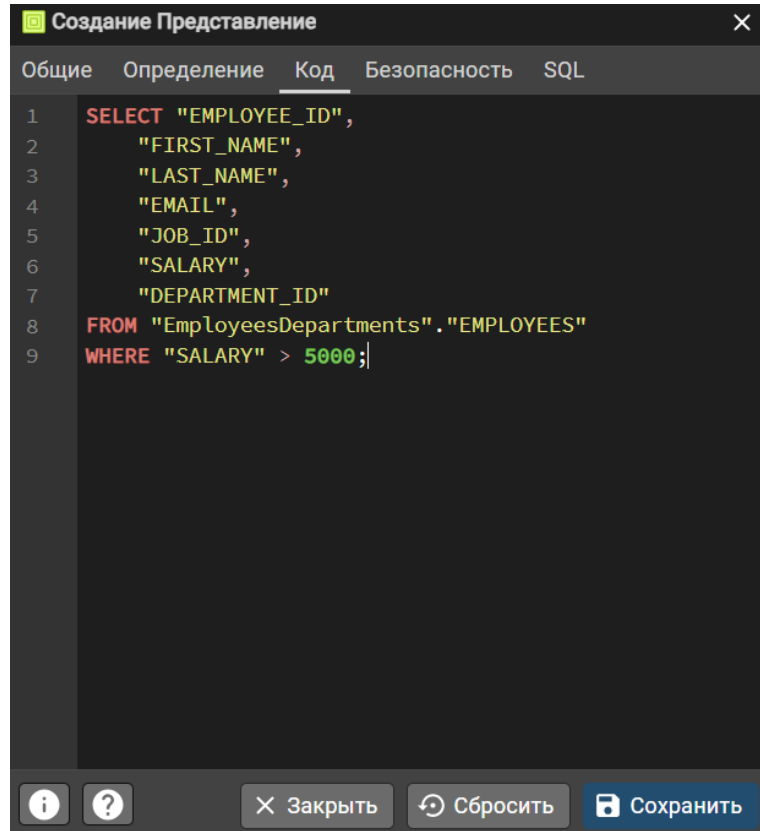


Рисунок 1 – Окно создания представления

На рисунке 2 представлен код представления.



```
1 SELECT "EMPLOYEE_ID",
2     "FIRST_NAME",
3     "LAST_NAME",
4     "EMAIL",
5     "JOB_ID",
6     "SALARY",
7     "DEPARTMENT_ID"
8 FROM "EmployeesDepartments"."EMPLOYEES"
9 WHERE "SALARY" > 5000;
```

Рисунок 2 – Код представления

На рисунке 3 представлен вывод представления.

Запрос

История запросов

1

SELECT * FROM "EmployeesDepartments".v_emp_active

2

</

Рисунок 3 – Вывод представления

2 Задание 2

На рисунке 4 представлен SQL-запрос для создания представления на основе другого представления.

```

Запрос  История запросов
1  CREATE OR REPLACE VIEW "EmployeesDepartments"."V_EMP_ACTIVE_INFO" AS
2  SELECT "EMPLOYEE_ID",
3         "FIRST_NAME" || ' ' || "LAST_NAME" AS FULL_NAME,
4         "EMAIL",
5         "JOB_ID",
6         "SALARY",
7         "DEPARTMENT_ID"
8  FROM "EmployeesDepartments".v_emp_active
9  WHERE "DEPARTMENT_ID" = 80;

```

Рисунок 4 – SQL-запрос

На рисунке 5 представлен вывод представления.

Запрос

История запросов

1

SELECT * FROM "EmployeesDepartments"."V_EMP_ACTIVE_INFO"

Data Output

Сообщения

Notifications

SQL

Showing rows: 1 to 4

	EMPLOYEE_ID integer	full_name text	EMAIL character varying (25)	JOB_ID character varying (10)	SALARY numeric (8,2)	DEPARTMENT_ID smallint
1	149	Eleni Zlotkey	EZLOTKEY	SA_MAN	10500.00	80
2	174	Ellen Abel	EABEL	SA_REP	11000.00	80
3	176	Jonathon Tayl...	JTAYLOR	SA_REP	8600.00	80
4	212	Nick Hooper	NHOOPER	SR_SA_REP	9600.00	80

Рисунок 5 – Вывод представления

На рисунке 6 представлено добавление данных в представление двумя способами.

Запрос

История запросов

1

INSERT INTO "EmployeesDepartments"."EMPLOYEES"

2

("FIRST_NAME", "LAST_NAME", "EMAIL", "JOB_ID", "SALARY", "DEPARTMENT_ID")

3

VALUES('Ivan', 'Ivanov', 'Iv_Ivn@itmo.ru', 'SA_REP', 6000, 80);

4

5

INSERT INTO "EmployeesDepartments".v_emp_active

6

("FIRST_NAME", "LAST_NAME", "EMAIL", "JOB_ID", "SALARY", "DEPARTMENT_ID")

7

VALUES('Petr', 'Petrov', 'Pe_Ptr@itmo.ru', 'AD_VP', 22000, 80);

8

9

SELECT * FROM "EmployeesDepartments"."V_EMP_ACTIVE_INFO";

Data Output

Сообщения

Notifications

</

Рисунок 6 – Добавление данных

На рисунке 7 представлена попытка редактирования таблицы. Выводится ошибка, потому что таблица задействована в представлении.

Запрос	История запросов
1	ALTER TABLE "EmployeesDepartments"."EMPLOYEES"
2	ALTER COLUMN "FIRST_NAME" TYPE varchar(100);

Data Output	Сообщения	Notifications
ERROR: изменить тип столбца, задействованного в представлении или правиле, нельзя правило _RETURN для отношения представление "EmployeesDepartments".v_emp_active зависит от столбца "FIRST_NAME"		
ОШИБКА: изменить тип столбца, задействованного в представлении или правиле, нельзя SQL-состояние: 0A000 Подробности: правило _RETURN для отношения представление "EmployeesDepartments".v_emp_active зависит от столбца "FIRST_NAME"		

Рисунок 7 – Попытка редактирования таблицы

3 Задание 3

На рисунке 8 представлен SQL-запрос создания материализованного представления.

ЗапросИстория запросов

1

CREATE MATERIALIZED VIEW "EmployeesDepartments".v_emp_active AS
SELECT "EMPLOYEE_ID",
"FIRST_NAME",
"LAST_NAME",
"EMAIL",
"JOB_ID",
"SALARY",
"DEPARTMENT_ID"
FROM "EmployeesDepartments"."EMPLOYEES"
WHERE "SALARY" > 5000;

23

Data OutputСообщенияNotifications

SELECT 26

Запрос завершён успешно, время выполнения: 47 мсек.

Рисунок 8 – Материализованное представление

На рисунке 9 представлен вывод Иванова из таблицы.

ЗапросИстория запросов

1

SELECT "EMPLOYEE_ID",
"FIRST_NAME",
"LAST_NAME",
"EMAIL",
"JOB_ID",
"SALARY",
"DEPARTMENT_ID"
FROM "EmployeesDepartments"."EMPLOYEES"
WHERE "LAST_NAME" = 'Ivanov'

23

Data OutputСообщенияNotifications

Showing rows: 1 to 1Page No: 1

	EMPLOYEE_ID [PK] integer	FIRST_NAME character varying (20)	LAST_NAME character varying (25)	EMAIL character varying (25)	JOB_ID character varying (10)	SALARY numeric (8,2)	DEPARTMENT_ID smallint
1	236	Ivan	Ivanov	Iv_lvn@itmo.ru	SA_REP	6000.00	80

Рисунок 9 – Вывод из таблицы

На рисунке 10 представлено обновление данных в таблице.

ЗапросИстория запросов

1

UPDATE "EmployeesDepartments"."EMPLOYEES"
SET "FIRST_NAME" = 'Petr'
WHERE "LAST_NAME" = 'Ivanov';

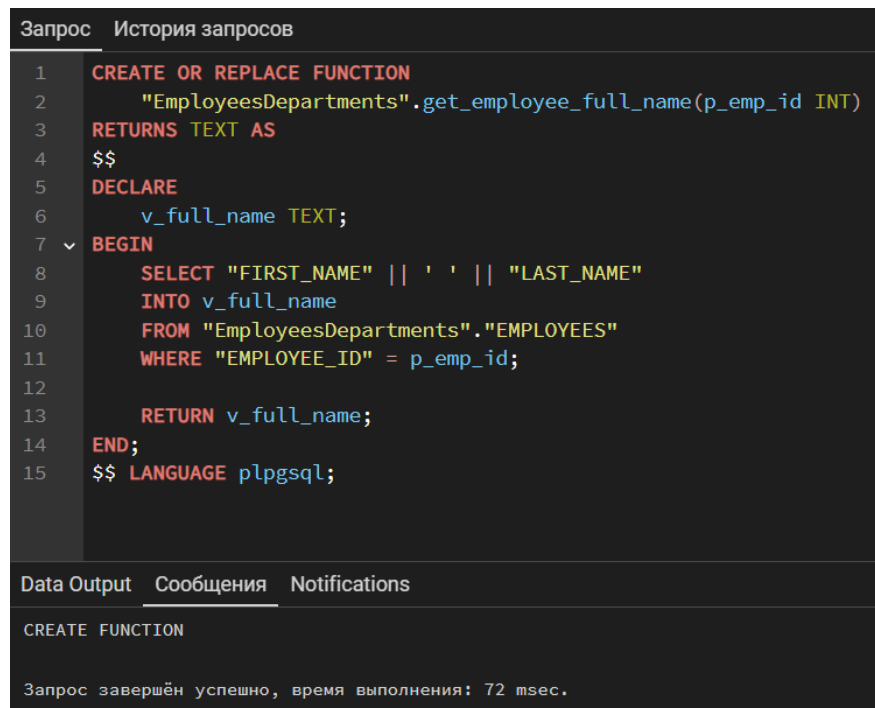
23

Data OutputСообщенияNotifications

Showing rows: 1 to 1Page No: 1 of 1

	EMPLOYEE_ID [PK] integer	FIRST_NAME character varying (20)	LAST_NAME character varying (25)	EMAIL character varying (25)	PHONE_NUMBER character varying (20)	HIRE_DATE date	JOB_ID character varying (10)	SALARY numeric (8,2)
1	236	Petr	Ivanov	Iv_lvn@itmo.ru	[null]	2025-12-14	SA_REP	6000.00

Рисунок 10 – Обновление данных



```

1 CREATE OR REPLACE FUNCTION
2   "EmployeesDepartments".get_employee_full_name(p_emp_id INT)
3 RETURNS TEXT AS
4 $$
5 DECLARE
6   v_full_name TEXT;
7 BEGIN
8   SELECT "FIRST_NAME" || ' ' || "LAST_NAME"
9   INTO v_full_name
10  FROM "EmployeesDepartments"."EMPLOYEES"
11  WHERE "EMPLOYEE_ID" = p_emp_id;
12
13  RETURN v_full_name;
14 END;
15 $$ LANGUAGE plpgsql;

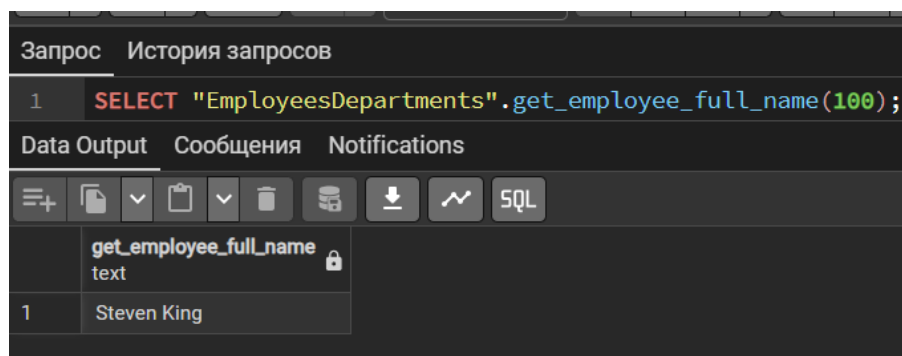
```

CREATE FUNCTION

Запрос завершён успешно, время выполнения: 72 мсек.

Рисунок 13 – Создание функции

На рисунке 14 представлен вызов функции.



```

1 SELECT "EmployeesDepartments".get_employee_full_name(100);

```

	get_employee_full_name text
1	Steven King

Рисунок 14 – Вызов функции

5 Задание 5

На рисунке 15 представлена созданная функция. Функция получает на вход ID сотрудника, и зарплату, которая считается в организации высокой. Если зарплата сотрудника больше высокой, то функция возвращает “высокая зарплата”, если меньше – “низкая”.

```

CREATE OR REPLACE FUNCTION
  "EmployeesDepartments".is_high_salary(p_emp_id INT, high_sal DECIMAL)
RETURNS TEXT AS
$$
DECLARE
  v_salary DECIMAL;
BEGIN
  SELECT "SALARY"
  INTO v_salary
  FROM "EmployeesDepartments"."EMPLOYEES"
  WHERE "EMPLOYEE_ID" = p_emp_id;

  IF NOT FOUND THEN
    RETURN 'Сотрудник не найден';
  END IF;

  IF v_salary >= high_sal THEN
    RETURN 'Высокая зарплата';
  ELSE
    RETURN 'Обычная зарплата';
  END IF;
END;
$$ LANGUAGE plpgsql;

```

Рисунок 15 – Созданная функция

На рисунке 16 представлен первый вариант использования функции. В данном случае, зарплата сотрудника оказалась высокой.

Запрос		История запросов		
1	SELECT			
2	"EMPLOYEE_ID",			
3	"LAST_NAME",			
4	"SALARY",			
5	"EmployeesDepartments".is_high_salary("EMPLOYEE_ID", 10000) AS "Результат"			
6	FROM "EmployeesDepartments"."EMPLOYEES"			
7	WHERE "EMPLOYEE_ID" = 100;			
Data Output		Сообщения		
		Notifications		
		Showing rows: 1		
	EMPLOYEE_ID [PK] integer	LAST_NAME character varying (25)	SALARY numeric (8,2)	Результат text
1	100	King	24000.00	Высокая зарпла...

Рисунок 16 – Использование функции

На рисунке 17 представлен второй вариант использования функции. В данном случае, зарплата сотрудника оказалась низкой.

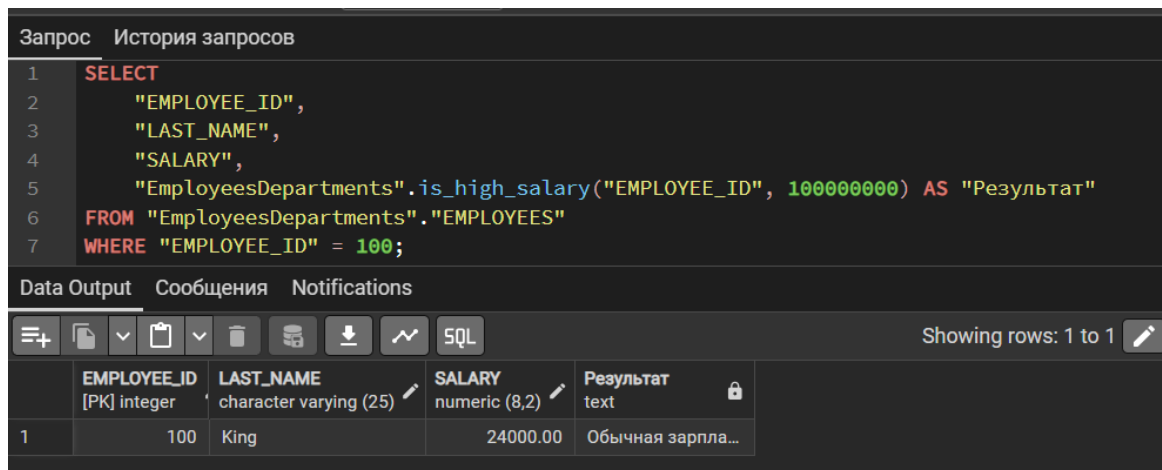


Рисунок 17 – Использование функции

6 Задание 6

На рисунке 18 представлено создание процедуры.

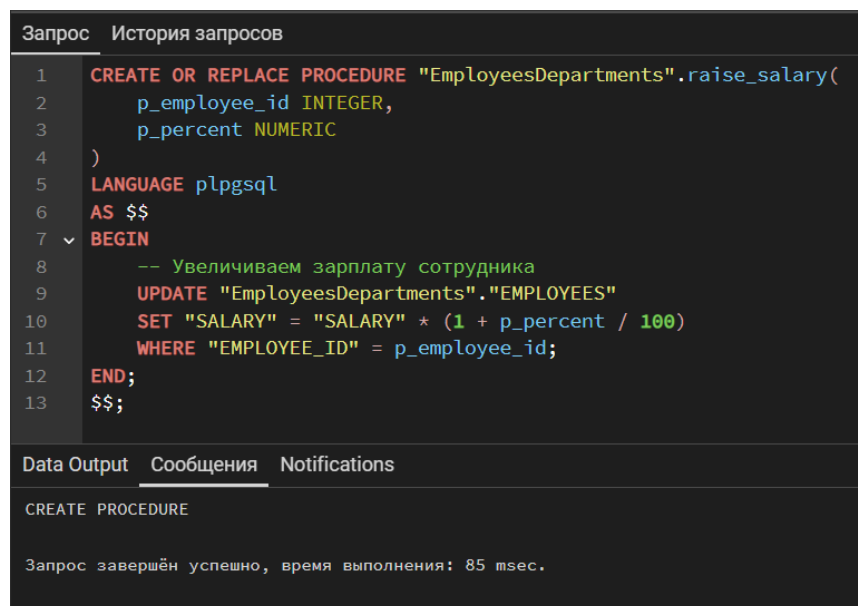


Рисунок 18 – Создание процедуры

На рисунке 19 представлена текущая зарплата сотрудника.

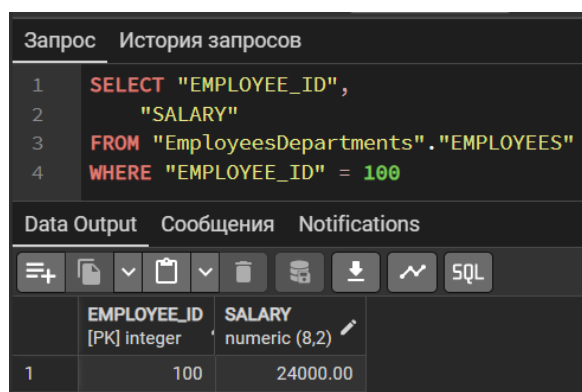


Рисунок 19 – Текущая зарплата

На рисунке 20 представлены вызов процедуры и новая зарплата сотрудника.

Запрос

История запросов

1

2

3

4

5

6

CALL "EmployeesDepartments".raise_salary(100, 10);

SELECT "EMPLOYEE_ID",

"SALARY"

FROM "EmployeesDepartments"."EMPLOYEES"

WHERE "EMPLOYEE_ID" = 100

Data Output

Сообщения

Notifications

≡+

▼

▼

SQL

EMPLOYEE_ID

SALARY

[PK] integer

numeric (8,2)

1

100

26400.00

Рисунок 20 – Вызов процедуры и результат

7 Задание 7

На рисунке 21 представлена созданная процедура. Она может выполнять два действия: повысить зарплату на 10% или добавить бонус.

Запрос	История запросов
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 <pre>CREATE OR REPLACE PROCEDURE "EmployeesDepartments".my_procedure(emp_id INTEGER, todo TEXT) LANGUAGE plpgsql AS \$\$ BEGIN IF NOT EXISTS (SELECT * FROM "EmployeesDepartments"."EMPLOYEES" WHERE "EMPLOYEE_ID" = emp_id) THEN RETURN; END IF; IF todo = 'RAISE' THEN UPDATE "EmployeesDepartments"."EMPLOYEES" SET "SALARY" = "SALARY" * 1.10 WHERE "EMPLOYEE_ID" = emp_id; ELSIF todo = 'BONUS' THEN UPDATE "EmployeesDepartments"."EMPLOYEES" SET "BONUS" = '1000' WHERE "EMPLOYEE_ID" = emp_id; END IF; END; \$\$;</pre>	
Data Output	Сообщения
CREATE PROCEDURE	

Рисунок 21 – Созданная процедура

На рисунке 22 представлена зарплата и бонус сотрудника с id = 100 до вызовов процедуры.

The screenshot shows a SQL IDE interface. The top tab is 'Запрос' (Query). The query text is as follows:

```
1 SELECT "EMPLOYEE_ID", "LAST_NAME", "SALARY", "BONUS"
2 FROM "EmployeesDepartments"."EMPLOYEES"
3 WHERE "EMPLOYEE_ID" = 100;
4
5 --CALL "EmployeesDepartments".my_procedure(100, 'RAISE');
6
7 --SELECT "EMPLOYEE_ID", "LAST_NAME", "SALARY", "BONUS"
8 --FROM "EmployeesDepartments"."EMPLOYEES"
9 --WHERE "EMPLOYEE_ID" = 106;
```

Below the query editor, there are tabs for 'Data Output', 'Сообщения' (Messages), and 'Notifications'. The 'Data Output' tab is active, showing a table with the following data:

	EMPLOYEE_ID [PK] integer	LAST_NAME character varying (25)	SALARY numeric (8,2)	BONUS character varying (5)
1	100	King	26400.00	[null]

Рисунок 22 – До процедуры

На рисунке 23 представлен вызов процедуры для поднятия зарплаты на 10% и результат.

The screenshot shows a SQL IDE interface. The top tab is 'Запрос' (Query). The query text is as follows:

```
1 CALL "EmployeesDepartments".my_procedure(100, 'RAISE');
2
3 SELECT "EMPLOYEE_ID", "LAST_NAME", "SALARY", "BONUS"
4 FROM "EmployeesDepartments"."EMPLOYEES"
5 WHERE "EMPLOYEE_ID" = 100;
```

Below the query editor, there are tabs for 'Data Output', 'Сообщения' (Messages), and 'Notifications'. The 'Data Output' tab is active, showing a table with the following data:

	EMPLOYEE_ID [PK] integer	LAST_NAME character varying (25)	SALARY numeric (8,2)	BONUS character varying (5)
1	100	King	29040.00	[null]

Рисунок 23 – Вызов процедуры и результат

На рисунке 24 представлен вызов процедуры для добавления бонуса в размере 1000 и результат.

The screenshot shows a database IDE interface. At the top, there are two tabs: "Запрос" (Query) and "История запросов" (Query History). The "Запрос" tab is active, displaying a SQL script with line numbers 1 through 5. The script consists of a call to a procedure followed by a select statement. Below the script, there are three tabs: "Data Output", "Сообщения" (Messages), and "Notifications". The "Data Output" tab is active, showing a table of results. The table has five columns: "EMPLOYEE_ID" (integer), "LAST_NAME" (character varying), "SALARY" (numeric), and "BONUS" (character varying). A single row of data is displayed, corresponding to employee ID 100, last name King, salary 29040.00, and bonus 1000.

```
1 CALL "EmployeesDepartments".my_procedure(100, 'BONUS');
2
3 SELECT "EMPLOYEE_ID", "LAST_NAME", "SALARY", "BONUS"
4 FROM "EmployeesDepartments"."EMPLOYEES"
5 WHERE "EMPLOYEE_ID" = 100;
```

	EMPLOYEE_ID [PK] integer	LAST_NAME character varying (25)	SALARY numeric (8,2)	BONUS character varying (5)
1	100	King	29040.00	1000

Рисунок 24 – Вызов процедуры и результат

ЗАКЛЮЧЕНИЕ

В ходе данной лабораторной работы удалось получить знания о представлениях в SQL, а также их разновидностях: обычное и материальное. Были получены знания об их различиях. Также эти знания были применены на практике в ходе выполнения работы.

Помимо представлений, удалось познакомиться с функциями и процедурами, разницей между ними. Также были созданы собственные функция и процедура для применения знаний о них на практике.

В ходе выполнения данной лабораторной работы возникали трудности при создании своих функции и процедуры из-за недостатка опыта работы с языком SQL. Эту проблему удалось решить, ознакомившись с справочным материалом в интернете, в результате чего функция и процедура были созданы успешно.